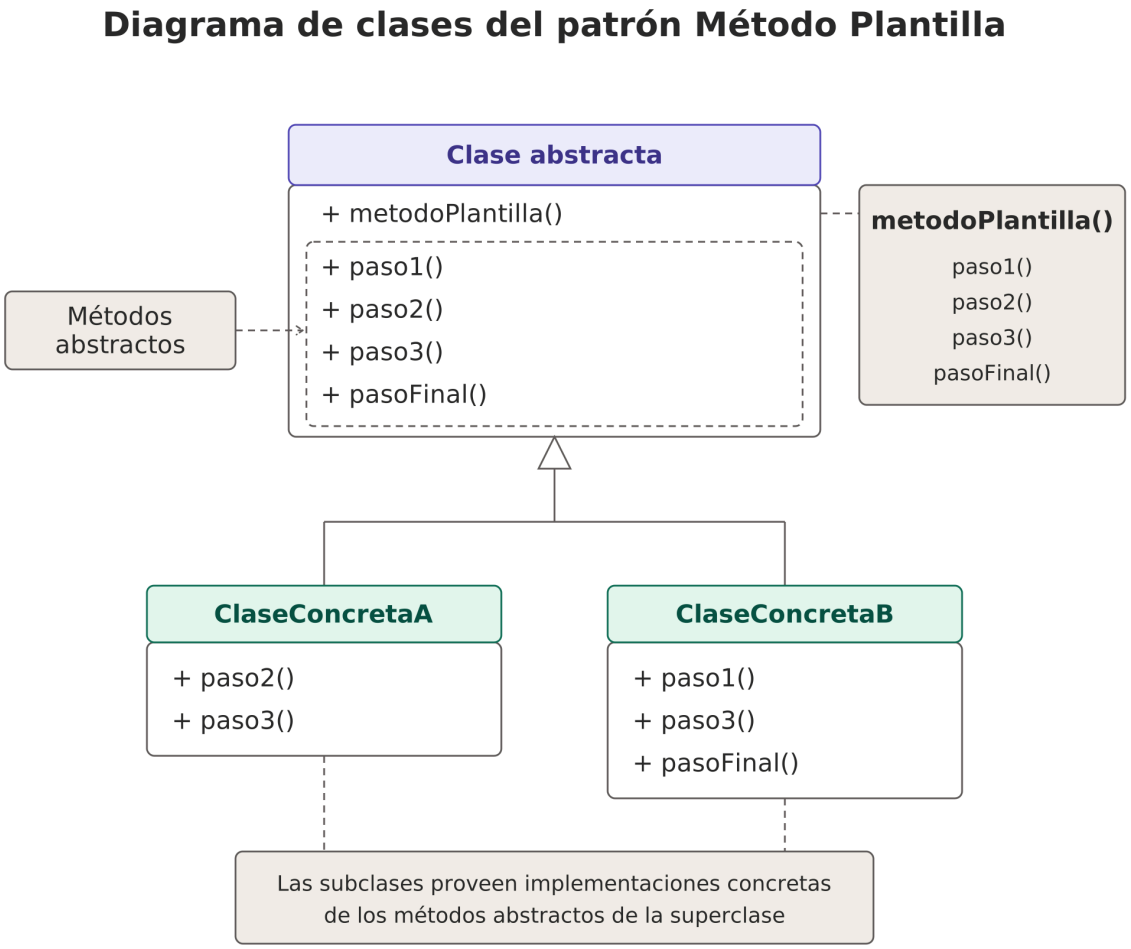
b) Investigación:

- Patrón Plantilla (Template Method):

  ➢ ¿Qué especifica el patrón plantilla? Realizar el Diagrama de Clases. ¿Cómo se utilizaría el en las clases Controlador y clases de Servicio?

  El patrón plantilla o template method es un patrón de diseño de comportamiento que define el esqueleto de un algoritmo en la superclase pero permite que las subclases sobreescriban pasos del algoritmo sin cambiar su estructura.



  En las clases Controlador y de Servicio se utiliza este patrón sobre todo para las operaciones de CRUD ya que se repite la misma secuencia de pasos cambiando algunos detalles como lo pueden ser las validaciones específicas de cada clase, etc. Ejemplo en código de cómo se vería una clase ServicioBase para poder ilustrarlo mejor:

```java
public abstract class ServicioBase<T, ID> {

    public final T guardar(T entidad) {
        validar(entidad);
        antesDeGuardar(entidad);
        T guardada = persistir(entidad);
        despuesDeGuardar(guardada);
        return guardada;
    }

    public abstract T buscarPorId(ID id);
    protected abstract void validar(T entidad);
    protected abstract T persistir(T entidad);
    protected void antesDeGuardar(T entidad) { }
    protected void despuesDeGuardar(T entidad) { }
}

public class ClienteServicio extends ServicioBase<Cliente, Long> {
    @Override protected void validar(Cliente c) { /* ... */ }
    @Override protected Cliente persistir(Cliente c) { /* ... */ return c; }
    @Override public Cliente buscarPorId(Long id) { /* ... */ return null; }
}
```

La clase ControladorBase quedaría bastante parecida, con un método plantilla procesar(request) que ejecuta autenticar → validarParametros → ejecutar → armarRespuesta. Cada controlador concreto implementa solo ejecutar() y lo propio de su entidad.

- Generics:

  ➢ ¿Qué es Java Generics y cuáles son sus ventajas? ¿Cómo podría

  complementarse con el patrón plantilla para generalizar las clases

  Controlador y clases de Servicio?

  Es un mecanismo que permite parametrizar clases, interfaces y métodos con tipos (<T>), de modo que el mismo código funcione con distintos tipos manteniendo el chequeo de tipos en compilación, dicho de otro modo, los genéricos permiten escribir clases, interfaces y métodos que funcionan con distintos tipos de datos, sin necesidad de especificar el tipo exacto de antemano.

  Se podrían complementar con el patrón plantilla creando una clase Base ya sea de tipo Controlador o Servicio genérica para todas las entidades que sean parte de nuestro proyecto y así poder implementar las operaciones comunes a todas en una sola clase reduciendo así la cantidad de código duplicado en nuestro proyecto.

Además al concentrar la lógica común a todas las clases en una sola el mantenimiento luego es mucho más sencillo. Esto lo podemos ver ilustrado en el código del ejemplo anterior.

- Reflection:

  ➢ ¿Qué es Java Reflection y cuáles son sus ventajas? Realizar en código de

  cómo se accede a los atributos privados de una clase y como se puede

  invocar un método.

  Es la API que permite que un programa inspeccione y modifique su propia estructura en tiempo de ejecución: obtener las clases, atributos, métodos y constructores, leer o escribir campos, e invocar métodos, sin conocerlos en tiempo de compilación.

  Ventajas

  - Permite código dinámico y genérico: instanciar clases o invocar métodos a partir de su nombre (por ejemplo, desde un archivo de configuración).
  - Es la base de frameworks como Spring (inyección de dependencias), Hibernate/JPA (mapeo de entidades), JUnit (descubrimiento de tests) y librerías de serialización como Jackson.
  - Facilita herramientas de análisis, depuradores y generadores de código.